Guía de referencia de pandas

Funciones, métodos y atributos principales, con una explicación de lo que hace cada uno

Referencia basada en pandas 3.0 · Octubre de 2026

Cómo leer esta guía: cada tabla tiene a la izquierda el nombre (con sus parámetros más importantes) y a la derecha lo que hace. Los textos entre comillas angulares, como «A1» o «records», son valores literales; «df» es un DataFrame, «s» una Series, «gb» un objeto groupby y «idx» un índice. El símbolo · separa elementos relacionados que se explican juntos.

Alcance: cubre la API pública de pandas (estructuras, lectura y escritura, selección, limpieza, agrupación, combinación, texto, fechas, ventanas, índices, gráficos, estilos y opciones). La lista de funciones de cada sección se verificó contra pandas 3.0.2. Si usás pandas 2.x, algunas cosas cambian: la sección 29 resume las diferencias.

Contenido

1. Objetos y conceptos principales 4
2. Funciones de nivel superior (pd.*) 5
3. Lectura de datos (read_*) 6
4. Escritura de datos (to_*) 9
5. Atributos de Series y DataFrame 11
6. Inspección rápida de los datos 12
7. Selección e indexación 13
8. Agregar, quitar, renombrar y reordenar 14
9. Tipos de datos (dtypes) y conversión 16
10. Valores faltantes 18
11. Duplicados, orden y ranking 19
12. Aritmética, comparación y lógica 19
13. Estadísticas y agregaciones 21
14. Aplicar funciones 21
15. Agrupar datos (groupby) 22
16. Combinar tablas 23
17. Reorganizar y transformar la forma de los datos 24
18. Series: métodos propios y accesores 25
19. Texto: accesor .str 26
20. Fechas: accesor .dt 28
21. Categorías: accesor .cat 29
22. Fechas y series de tiempo 29
23. Ventanas móviles (rolling, expanding, ewm) 31
24. Index y MultiIndex 33
25. Visualización (plot) 36
26. Estilos visuales (df.style) 37
27. Opciones de configuración 39
28. Errores y advertencias (pd.errors) 40
29. Cambios importantes de pandas 3.0 40

30. Trampas frecuentes 41

1. Objetos y conceptos principales

pandas gira alrededor de dos estructuras: Series (una columna etiquetada) y DataFrame (una tabla de columnas con un índice de filas). Se importa por convención como «import pandas as pd».

Estructuras de datos

Elemento	Qué hace
`pd.Series(data=None, index=None, dtype=None, name=None, copy=None)`	Arreglo unidimensional con etiquetas (el índice). data puede ser lista, diccionario, ndarray o un valor escalar.
`pd.DataFrame(data=None, index=None, columns=None, dtype=None, copy=None)`	Tabla bidimensional. data puede ser un diccionario de listas o Series (clave = nombre de columna), una lista de diccionarios, una lista de listas, un ndarray u otro DataFrame.
`DataFrame.from_dict(data, orient=«columns», dtype=None, columns=None)`	Crea un DataFrame desde un diccionario. orient «columns»: las claves son columnas; «index»: las claves son filas; «tight»: formato compacto con índice y columnas.
`DataFrame.from_records(data, index=None, exclude=None, columns=None, coerce_float=False, nrows=None)`	Crea un DataFrame desde una lista de tuplas, registros o un arreglo estructurado.
`DataFrame.from_arrow(data)`	Crea un DataFrame desde una tabla de Apache Arrow (por ejemplo, de pyarrow).
`pd.array(data, dtype=None, copy=True)`	Crea un arreglo de pandas (extension array) con el tipo indicado, p. ej. enteros que admiten valores faltantes.
`pd.Index · pd.RangeIndex(start, stop, step)`	Etiquetas de filas o columnas. RangeIndex es el índice numérico 0, 1, 2... por defecto, y no ocupa memoria.
`pd.MultiIndex · pd.DatetimeIndex · pd.PeriodIndex · pd.TimedeltaIndex · pd.IntervalIndex · pd.CategoricalIndex`	Índices especializados: jerárquico (varios niveles), de fechas, de períodos, de duraciones, de intervalos y de categorías. Ver sección 24.
`pd.Categorical(values, categories=None, ordered=False)`	Arreglo categórico: valores tomados de un conjunto fijo de categorías, con orden opcional.

Valores escalares y especiales

Elemento	Qué hace
`pd.Timestamp · pd.Timedelta · pd.Period · pd.Interval · pd.DateOffset`	Escalares de pandas para un instante, una duración, un período (como «2026-10»), un intervalo y un desplazamiento de calendario. Ver sección 22.
`np.nan`	Valor faltante clásico de punto flotante (NaN). Es el marcador de faltante de las columnas numéricas y, en pandas 3, también de las de texto.
`pd.NA`	Valor faltante «oficial» de los tipos que admiten nulos (Int64, boolean, string...). Al comparar o operar da NA en lugar de True o False.
`pd.NaT`	Faltante para fechas y duraciones (Not a Time).
`pd.IndexSlice`	Ayuda para escribir rebanadas sobre un MultiIndex dentro de .loc, por ejemplo df.loc[pd.IndexSlice[:, «2026»], :].
`pd.NamedAgg(column, aggfunc)`	Define una agregación con nombre dentro de groupby().agg(). Equivale a escribir una tupla (columna, función).
`pd.Grouper(key=None, level=None, freq=None, sort=True, dropna=True)`	Especifica cómo agrupar: por una columna, un nivel del índice o una frecuencia de tiempo (p. ej. «MS» para mes).
`pd.col(nombre)`	Expresión que referencia una columna por su nombre, para usar dentro de assign o de selecciones sin repetir el nombre del DataFrame. Novedad de pandas 3.0.
`DataFrame.flags · Flags.allows_duplicate_labels`	Banderas del objeto; allows_duplicate_labels=False impide etiquetas repetidas en el índice o las columnas.

2. Funciones de nivel superior (pd.*)

Funciones que se llaman directamente desde el módulo, para crear, convertir o inspeccionar datos. Las de lectura (read_*) están en la sección 3 y las de combinar y reorganizar, en las secciones 16 y 17.

Elemento	Qué hace
`pd.to_numeric(arg, errors=«raise», downcast=None, dtype_backend)`	Convierte a número. errors=«coerce» convierte a NaN lo que no se pueda convertir; downcast reduce el tamaño del tipo («integer», «signed», «unsigned», «float»).
`pd.to_datetime(arg, errors, dayfirst, yearfirst, utc, format, exact, unit, origin, cache)`	Convierte texto, números o columnas a fechas. Ver sección 22.
`pd.to_timedelta(arg, unit=None, errors=«raise»)`	Convierte a duraciones (Timedelta).
`pd.isna(obj) · pd.isnull(obj)`	Indica qué valores son faltantes (NaN, NA, NaT, None). Sirve para escalares, Series y DataFrames; isnull es un alias.
`pd.notna(obj) · pd.notnull(obj)`	Lo contrario de isna.
`pd.unique(values)`	Valores únicos en orden de aparición.
`pd.factorize(values, sort=False, use_na_sentinel=True)`	Codifica los valores como números enteros y devuelve (códigos, valores únicos).
`pd.json_normalize(data, record_path=None, meta=None, meta_prefix=None, record_prefix=None, errors, sep=«.», max_level=None)`	Aplana JSON anidado (diccionarios dentro de diccionarios o listas) a un DataFrame. record_path indica qué lista expandir y meta qué campos repetir en cada fila.
`pd.eval(expr, ...)`	Evalúa una expresión de texto con operaciones sobre DataFrames o Series, de forma optimizada.

Elemento	Qué hace
`pd.date_range` · `pd.bdate_range` · `pd.period_range` · `pd.timedelta_range` · `pd.interval_range` · `pd.infer_freq`	Generadores de secuencias de tiempo e intervalos. Ver sección 22.
`pd.set_option` · `pd.get_option` · `pd.reset_option` · `pd.describe_option` · `pd.option_context` · `pd.set_eng_float_format`	Opciones de configuración. Ver sección 27.
`pd.to_pickle(obj, filepath_or_buffer, ...)`	Guarda cualquier objeto en formato pickle (función equivalente a to_pickle).
`pd.show_versions()`	Muestra la versión de pandas y de sus dependencias; útil al reportar un problema.
`pd.test()`	Ejecuta la batería de tests de pandas (requiere pytest).
`pd.api.types` · `pd.api.extensions` · `pd.api.indexers` · `pd.api.typing`	Submódulos de la API pública: funciones de verificación de tipos, extensión de pandas con tipos propios, indexadores de ventanas y clases para anotaciones de tipo.

3. Lectura de datos (read_*)

Todas devuelven un DataFrame (o una lista/diccionario de ellos, según el caso). Las más usadas tienen su detalle de parámetros.

Funciones de lectura

Elemento	Qué hace
`pd.read_csv(filepath_or_buffer, ...)`	Lee un archivo de texto delimitado (CSV) desde una ruta, una URL o un archivo abierto.
`pd.read_table(filepath_or_buffer, sep=«\t», ...)`	Como read_csv, pero con tabulación como separador por defecto.
`pd.read_excel(io, sheet_name=0, ...)`	Lee una hoja de un archivo Excel (.xlsx, .xls, .xlsm, .ods, .xlsb). Con sheet_name=None lee todas y devuelve un diccionario nombre-DataFrame.
`pd.read_json(path_or_buf, orient, typ, dtype, convert_dates, lines, ...)`	Lee JSON. orient define la forma del JSON («records», «split», «index», «columns», «values», «table»); lines=True lee un objeto JSON por línea.
`pd.read_html(io, match, flavor, header, index_col, ...)`	Lee todas las tablas HTML de una página o texto y devuelve una lista de DataFrames.
`pd.read_xml(path_or_buffer, xpath, ...)`	Lee datos de un archivo XML.
`pd.read_fwf(filepath_or_buffer, colspecs, widths, ...)`	Lee archivos de texto de ancho fijo.
`pd.read_sql(sql, con, index_col, coerce_float, params, parse_dates, columns, chunksize, dtype_backend, dtype)`	Lee desde una base de datos usando una consulta SQL o el nombre de una tabla. con es una conexión (SQLAlchemy o sqlite3).
`pd.read_sql_query(sql, con, ...)` · `pd.read_sql_table(table_name, con, ...)`	Variantes: una ejecuta una consulta; la otra lee una tabla completa (solo con SQLAlchemy).
`pd.read_parquet(path, engine=«auto», columns, filters, ...)`	Lee el formato columnar Parquet (requiere pyarrow o fastparquet). Permite leer solo ciertas columnas y filtrar filas.
`pd.read_feather(path, ...)` · `pd.read_orc(path, ...)`	Leen los formatos binarios Feather y ORC.
`pd.read_pickle(filepath_or_buffer, ...)`	Lee un objeto guardado con pickle. Solo usar con archivos de confianza.
`pd.read_hdf(path_or_buf, key, ...)`	Lee un almacén HDF5 (requiere PyTables).
`pd.read_stata(...)` · `pd.read_sas(...)` · `pd.read_spss(...)`	Leen archivos de Stata, SAS y SPSS.
`pd.read_clipboard(...)`	Lee el contenido del portapapeles como tabla.
`pd.read_iceberg(table_identifier, ...)`	Lee una tabla de Apache Iceberg.

Parámetros principales de read_csv

Elemento	Qué hace
`filepath_or_buffer`	Ruta del archivo, URL o archivo abierto.
`sep · delimiter`	Separador de campos (por defecto coma). Con engine=«python» puede ser una expresión regular.
`header`	Número de fila con los nombres de columna. Por defecto «infer» (primera fila). None indica que no hay encabezado.
`names`	Lista de nombres de columna a usar (sustituye o reemplaza al encabezado).
`index_col`	Columna o columnas que se usan como índice. False evita que la primera columna se tome como índice.
`usecols`	Columnas a leer (nombres, posiciones o una función). Reduce memoria y tiempo.
`dtype`	Tipo por columna, como un diccionario {columna: tipo}. Útil para forzar texto (p. ej. códigos con ceros a la izquierda).
`engine`	Motor de lectura: «c» (rápido), «python» (más flexible) o «pyarrow» (multihilo).
`converters`	Diccionario de funciones para transformar cada columna al leerla.
`true_values · false_values`	Textos que se interpretan como True y False.
`skipinitialspace`	Ignora espacios después del separador.
`skiprows · skipfooter · nrows`	Filas a saltar al inicio, filas a saltar al final y cantidad máxima de filas a leer.
`na_values · keep_default_na · na_filter`	Textos adicionales considerados faltantes, si se conservan los predeterminados («NA», «NaN», «», etc.) y si se detectan faltantes (False acelera la lectura).
`skip_blank_lines`	Salta las líneas vacías.
`parse_dates · date_format · dayfirst · cache_dates`	Columnas que se convierten en fechas, formato de fecha esperado, interpretar día antes que mes y uso de caché al convertir.
`iterator · chunksize`	Lectura por tandas: devuelve un lector que entrega DataFrames de chunksize filas, para archivos que no caben en memoria.
`compression`	Tipo de compresión («infer», «gzip», «zip», «bz2», «xz», «zstd», None).
`thousands · decimal`	Separador de miles y de decimales. Para formatos latinoamericanos suele ser thousands=«.» y decimal=«,».
`lineterminator · quotechar · quoting · doublequote · escapechar · comment · dialect`	Detalles del formato: fin de línea, carácter de comillas, política de comillas, comillas dobles, carácter de escape, carácter de comentario y dialecto csv.
`encoding · encoding_errors`	Codificación del archivo (p. ej. «utf-8», «latin-1», «cp1252») y qué hacer con caracteres inválidos.
`on_bad_lines`	Qué hacer con líneas con demasiados campos: «error», «warn», «skip» o una función.
`low_memory · memory_map · float_precision`	Procesamiento por bloques, mapeo del archivo en memoria y precisión de la lectura de decimales.
`storage_options · dtype_backend`	Opciones para almacenamiento remoto y tipo de respaldo («numpy_nullable» o «pyarrow»).

Parámetros principales de read_excel

Elemento	Qué hace
`io`	Ruta, URL, archivo abierto o un objeto ExcelFile.
`sheet_name`	Hoja a leer: nombre, posición (desde 0), lista de ambos o None para todas. Por defecto la primera.
`header · names · index_col · usecols`	Fila de encabezado, nombres de columna, columna de índice y columnas a leer (también acepta letras de Excel como «A:D»).
`dtype · converters · true_values · false_values`	Tipos por columna, funciones de conversión y textos interpretados como booleanos.
`engine`	Motor: «openpyxl» (.xlsx/.xlsm), «xlrd» (solo .xls), «odf» (.ods), «pyxlsb» (.xlsb) o «calamine» (rápido).
`skiprows · skipfooter · nrows`	Filas a saltar al inicio, al final y cantidad máxima a leer.
`na_values · keep_default_na · na_filter`	Manejo de valores faltantes.
`parse_dates · date_format`	Conversión de columnas a fechas y formato esperado.
`thousands · decimal · comment`	Separadores numéricos y carácter de comentario.
`engine_kwargs · storage_options · dtype_backend`	Opciones para el motor, almacenamiento remoto y tipo de respaldo.

Ojo: read_excel necesita que openpyxl esté instalado para leer .xlsx. Si una columna de códigos o documentos se lee como número y pierde ceros a la izquierda, forzá dtype=str en esa columna.

4. Escritura de datos (to_*)

Métodos de DataFrame (y de Series) que exportan los datos. Casi todos aceptan una ruta o archivo; si se omite, devuelven el resultado como texto.

Métodos de escritura

Elemento	Qué hace
`df.to_csv(path_or_buf=None, sep, na_rep, float_format, columns, header, index, index_label, mode, encoding, compression, quoting, quotechar, lineterminator, chunksize, date_format, doublequote, escapechar, decimal, errors, storage_options)`	Escribe CSV. index=False evita escribir el índice; na_rep define cómo se escriben los faltantes; decimal cambia el separador decimal; encoding fija la codificación; mode=«a» agrega al final.
`df.to_excel(excel_writer, sheet_name=«Sheet1», na_rep, float_format, columns, header, index, index_label, startrow, startcol, engine, merge_cells, inf_rep, freeze_panes, storage_options, engine_kwargs, autofilter)`	Escribe a Excel. startrow y startcol desplazan el inicio de la tabla; freeze_panes inmoviliza paneles; autofilter agrega filtros al encabezado; index=False evita escribir el índice. Para varias hojas se pasa un ExcelWriter.
`df.to_json(path_or_buf, orient, date_format, double_precision, force_ascii, date_unit, default_handler, lines, compression, index, indent, storage_options, mode)`	Escribe JSON. orient define la forma; lines=True escribe un objeto por línea; indent da formato legible.
`df.to_html(...) · df.to_xml(...) · df.to_latex(...)`	Exportan a tabla HTML, a XML y a código LaTeX.
`df.to_markdown(buf, mode, index, storage_options, **kwargs)`	Devuelve la tabla en formato Markdown (requiere tabulate).
`df.to_string(buf, columns, col_space, header, index, na_rep, formatters, float_format, justify, max_rows, max_cols, ...)`	Devuelve la tabla como texto plano, con control fino del formato.
`df.to_sql(name, con, schema, if_exists=«fail», index, index_label, chunksize, dtype, method)`	Escribe en una tabla SQL. if_exists: «fail» (error si existe), «replace» (reemplaza la tabla), «append» (agrega filas) o «delete_rows».
`df.to_parquet(path, engine, compression, index, partition_cols, storage_options, filesystem)`	Escribe Parquet; partition_cols crea una carpeta por valor de columna.
`df.to_feather(path, ...) · df.to_orc(path, ...)`	Escriben Feather y ORC.
`df.to_pickle(path, compression, protocol, storage_options)`	Guarda el objeto en formato pickle.
`df.to_hdf(path_or_buf, key, mode, complevel, complib, append, format, ...)`	Guarda en HDF5.
`df.to_stata(path, ...)`	Escribe un archivo de Stata.
`df.to_clipboard(excel=True, sep=None, ...)`	Copia la tabla al portapapeles, lista para pegar en Excel.
`df.to_dict(orient=«dict», into=dict, index=True)`	Convierte a diccionario. orient: «dict», «list», «series», «split», «tight», «records» (lista de diccionarios, uno por fila) o «index».
`df.to_records(index=True, column_dtypes=None, index_dtypes=None)`	Convierte a un arreglo de registros de NumPy.
`df.to_numpy(dtype=None, copy=False, na_value)`	Convierte a un arreglo de NumPy. Es la forma recomendada en lugar de df.values.

Elemento	Qué hace
`df.to_xarray() · df.to_iceberg(...)`	Convierte a un objeto xarray o escribe una tabla Iceberg.
`df.to_period(freq=None) · df.to_timestamp(freq, how)`	Convierten el índice entre fechas (Timestamp) y períodos (Period).
`Series.to_frame(name) · Series.to_list() · Series.tolist()`	De Series a DataFrame de una columna y a lista de Python.

Escritura a Excel con varias hojas (ExcelWriter y ExcelFile)

Elemento	Qué hace
`pd.ExcelWriter(path, engine=None, date_format=None, datetime_format=None, mode=«w», storage_options=None, if_sheet_exists=None, engine_kwargs=None)`	Administrador de contexto para escribir varias hojas en un mismo archivo: with pd.ExcelWriter(...) as w, y luego df.to_excel(w, sheet_name=...). mode=«a» agrega a un archivo existente.
`if_sheet_exists`	Qué hacer si la hoja ya existe (solo con mode=«a»): «error», «new» (crea otra con nombre distinto), «replace» (reemplaza) u «overlay» (escribe encima, conservando el resto de la hoja).
`writer.book · writer.sheets`	Acceso al objeto nativo del motor: con openpyxl, el Workbook y un diccionario nombre-hoja. Permite dar formato después de escribir con df.to_excel.
`writer.engine · writer.supported_extensions · writer.date_format · writer.datetime_format`	Motor en uso, extensiones admitidas y formatos de fecha y fecha-hora con que se escriben.
`writer.close() · writer.check_extension(ext)`	Cierra y guarda el archivo (automático con with); verifica que la extensión sea válida.
`pd.ExcelFile(path_or_buffer, engine=None, storage_options=None, engine_kwargs=None)`	Abre un libro de Excel una sola vez para leer varias hojas sin reabrir el archivo.
`excelfile.sheet_names`	Lista con los nombres de las hojas.
`excelfile.parse(sheet_name, header, names, index_col, usecols, ...)`	Lee una hoja y devuelve un DataFrame (acepta los mismos parámetros que read_excel).
`excelfile.book · excelfile.close()`	Objeto nativo del libro y cierre del archivo.

5. Atributos de Series y DataFrame

Propiedades que se consultan sin paréntesis.

DataFrame

Elemento	Qué hace
`df.shape`	Tupla (filas, columnas).
`df.size · df.ndim`	Cantidad total de elementos y cantidad de dimensiones (2).
`df.columns`	Índice con los nombres de las columnas. Se puede asignar una lista para renombrarlas todas.
`df.index`	Índice de filas.
`df.dtypes`	Tipo de dato de cada columna.
`df.values`	Contenido como arreglo de NumPy. Se prefiere df.to_numpy().
`df.T`	Transpuesta: filas y columnas intercambiadas.
`df.axes`	Lista [índice de filas, índice de columnas].
`df.empty`	True si no tiene filas o columnas.
`df.attrs`	Diccionario libre para guardar metadatos propios del objeto.
`df.flags`	Banderas como allows_duplicate_labels.
`df.loc · df.iloc · df.at · df.iat`	Accesores de selección por etiqueta, por posición y de un solo valor (sección 7).
`df.style`	Devuelve un Styler para dar formato visual (sección 26).
`df.plot`	Accesor de gráficos (sección 25).
`df.sparse`	Accesor para columnas dispersas (sparse).

Series

Elemento	Qué hace
`s.name`	Nombre de la Series (se vuelve nombre de columna al pasarla a DataFrame).
`s.dtype`	Tipo de dato.
`s.array · s.values`	Contenido como arreglo de pandas o de NumPy.
`s.shape · s.size · s.ndim`	Forma (n,), cantidad de elementos y dimensiones (1).
`s.index`	Etiquetas de la Series.
`s.hasnans`	True si contiene algún valor faltante.
`s.is_unique`	True si no hay valores repetidos.
`s.is_monotonic_increasing · s.is_monotonic_decreasing`	True si los valores están ordenados de menor a mayor o de mayor a menor.
`s.nbytes`	Bytes que ocupan los datos.
`s.str · s.dt · s.cat · s.list · s.struct · s.sparse`	Accesores especializados: texto, fechas, categorías, listas y estructuras (tipos Arrow) y datos dispersos.

6. Inspección rápida de los datos

Primeros métodos que conviene usar al cargar una tabla nueva.

Elemento	Qué hace
`df.head(n=5) · df.tail(n=5)`	Primeras o últimas n filas.
`df.info(verbose, buf, max_cols, memory_usage, show_counts)`	Resumen: cantidad de filas, columnas, tipo de cada una, cantidad de valores no nulos y memoria usada.
`df.describe(percentiles=None, include=None, exclude=None)`	Estadísticas descriptivas (cantidad, media, desvío, mínimo, cuartiles, máximo). Por defecto solo columnas numéricas; include=«all» incluye todas.
`df.sample(n=None, frac=None, replace=False, weights=None, random_state=None, axis=None, ignore_index=False)`	Muestra aleatoria de n filas o de una fracción. random_state fija la semilla para obtener el mismo resultado.
`df.memory_usage(index=True, deep=False)`	Memoria por columna en bytes. deep=True mide también el contenido de las columnas de texto.
`df.nunique(axis=0, dropna=True)`	Cantidad de valores distintos por columna.
`df.value_counts(subset=None, normalize=False, sort=True, ascending=False, dropna=True)`	Cuenta cuántas veces aparece cada combinación de filas. En una Series cuenta cada valor; normalize=True da proporciones.
`df.count(axis=0, numeric_only=False)`	Cantidad de valores no nulos por columna.
`df.isna() · df.notna()`	DataFrame de True/False según haya o no valores faltantes. Se combina con sum() para contarlos por columna.
`df.duplicated(subset=None, keep=«first»)`	Marca las filas repetidas.
`df.equals(other)`	True si dos objetos tienen los mismos valores, forma y tipos.
`df.select_dtypes(include=None, exclude=None)`	Selecciona columnas según su tipo («number», «object», «str», «category», «datetime», «bool», etc.).
`df.first_valid_index() · df.last_valid_index()`	Etiqueta de la primera y de la última fila con algún valor no nulo.
`len(df)`	Cantidad de filas.
`df.keys() · df.items()`	Nombres de columnas; pares (nombre, Series) por cada columna.
`df.copy(deep=True)`	Copia independiente del objeto.
`Series.unique() · Series.mode()`	Valores únicos y valores más frecuentes.

7. Selección e indexación

Formas de elegir filas, columnas y valores. Las máscaras booleanas se combinan con & (y), | (o) y ~ (no), siempre con cada condición entre paréntesis.

Con corchetes y accesores

Elemento	Qué hace
`df[«col»] · df[[«a», «b»]]`	Una columna (devuelve Series) o varias columnas (devuelve DataFrame).
`df[mascara]`	Filas donde la condición es True, p. ej. df[df[«edad»] > 30].
`df[0:3]`	Cortar con rebanada selecciona filas por posición (las primeras 3).
`df.loc[filas, columnas]`	Selección por etiqueta. Acepta etiqueta, lista, rebanada (el extremo final está incluido) o máscara booleana. También sirve para asignar valores.
`df.iloc[filas, columnas]`	Selección por posición entera. En las rebanadas el extremo final no está incluido.
`df.at[fila, col] · df.iat[fila, col]`	Acceso rápido a un solo valor por etiqueta (at) o por posición (iat).
`df.get(key, default=None)`	Como df[key], pero devuelve default si la columna no existe en lugar de dar error.
`df.xs(key, axis=0, level=None, drop_level=True)`	Corte transversal: selecciona por una clave, ideal con índices jerárquicos.
`df.take(indices, axis=0)`	Selecciona por posiciones, dadas como lista.
`df.filter(items=None, like=None, regex=None, axis=None)`	Selecciona filas o columnas por nombre: lista exacta (items), texto contenido (like) o expresión regular (regex).
`df.query(expr, parser, engine, local_dict, global_dict, resolvers, level, inplace)`	Filtra filas con una expresión de texto, p. ej. «edad > 30 and ciudad == @ciudad». Con @ se usan variables de Python.
`df.isin(values) · Series.isin(values)`	True donde el valor pertenece a la lista dada (o diccionario, en DataFrame).
`Series.between(left, right, inclusive=«both»)`	True si el valor está entre dos límites; inclusive: «both», «neither», «left» o «right».
`df.where(cond, other, inplace, axis, level)`	Conserva los valores donde cond es True y reemplaza el resto por other (NaN por defecto).
`df.mask(cond, other, inplace, axis, level)`	Lo opuesto de where: reemplaza donde cond es True.
`df.nlargest(n, columns, keep=«first») · df.nsmallest(n, columns, keep=«first»)`	Las n filas con mayores o menores valores en las columnas indicadas.
`df.idxmax(axis=0, skipna=True, numeric_only=False) · df.idxmin(...)`	Etiqueta del índice donde está el máximo o el mínimo de cada columna.
`df.truncate(before=None, after=None, axis=None)`	Recorta filas antes o después de una etiqueta (índice ordenado).
`df.between_time(start, end, inclusive) · df.at_time(time)`	Con índice de fechas: filas dentro de un rango horario o a una hora exacta.
`df.squeeze(axis=None)`	Convierte a Series o escalar cuando solo hay una columna o un valor.
`df.head() · df.tail() · df.sample()`	También sirven para tomar subconjuntos (sección 6).

Asignar en una selección encadenada como df[df.a > 0][«b»] = 1 no modifica df en pandas 3. Para modificar valores usá una sola operación: df.loc[df.a > 0, «b»] = 1.

8. Agregar, quitar, renombrar y reordenar

Modificaciones de estructura. La mayoría devuelve un objeto nuevo; inplace=True modifica el original y devuelve None (se desaconseja, no suele ahorrar memoria).

Columnas y filas

Elemento	Qué hace
`df[«nueva»] = valores`	Crea o reemplaza una columna con un valor, lista, Series o resultado de una operación.
`df.assign(**kwargs)`	Devuelve un DataFrame con nuevas columnas, pasadas como nombre=valor o nombre=función. Permite encadenar pasos.
`df.insert(loc, column, value, allow_duplicates)`	Inserta una columna en una posición concreta (modifica el original).
`df.drop(labels=None, axis=0, index=None, columns=None, level=None, inplace=False, errors=«raise»)`	Elimina filas o columnas por nombre. Es más claro con columns=[...] o index=[...]. errors=«ignore» tolera nombres inexistentes.
`df.pop(«col») · del df[«col»]`	pop elimina la columna y la devuelve; del solo la elimina.
`df.dropna(...) · df.drop_duplicates(...)`	Eliminan filas con faltantes o repetidas (secciones 10 y 11).
`df.explode(column, ignore_index=False)`	Convierte cada elemento de una lista en su propia fila, repitiendo el resto de los datos.
`df.update(other, join=«left», overwrite=True, filter_func=None, errors=«ignore»)`	Actualiza los valores con los de otro DataFrame, alineando por etiquetas (modifica el original).
`df.isetitem(loc, value)`	Reemplaza por completo la columna de una posición dada, sin intentar escribir en el arreglo existente.

Nombres e índice

Elemento	Qué hace
`df.rename(mapper=None, index=None, columns=None, axis=None, inplace=False, level=None, errors=«ignore»)`	Cambia etiquetas con un diccionario o función. Lo habitual es rename(columns={«viejo»: «nuevo»}).
`df.rename_axis(mapper, index, columns, axis)`	Cambia el nombre del eje (del índice o del conjunto de columnas), no sus etiquetas.
`df.set_axis(labels, axis=0)`	Reemplaza todas las etiquetas de un eje por una lista nueva.
`df.add_prefix(prefix)` · `df.add_suffix(suffix)`	Agrega un texto al principio o al final de cada nombre de columna.
`df.set_index(keys, drop=True, append=False, inplace=False, verify_integrity=False)`	Usa una o varias columnas como índice.
`df.reset_index(level=None, drop=False, inplace=False, col_level=0, col_fill=«», allow_duplicates, names)`	Devuelve el índice a una columna normal y crea uno nuevo 0, 1, 2... drop=True descarta el índice viejo.
`df.reindex(labels=None, index=None, columns=None, axis=None, method=None, level=None, fill_value=nan, limit=None, tolerance=None)`	Ajusta el objeto a un conjunto nuevo de etiquetas: agrega las que faltan (con NaN o fill_value) y descarta las que sobran. También reordena.
`df.reindex_like(other)`	Reindexa para que coincida con el índice y las columnas de otro objeto.
`df.align(other, join=«outer», axis=None, level=None, fill_value=None)`	Alinea dos objetos por sus etiquetas y devuelve ambos ya alineados.
`df.droplevel(level, axis=0)` · `df.reorder_levels(order, axis=0)` · `df.swaplevel(i, j, axis=0)`	Quitan, reordenan o intercambian niveles de un índice jerárquico.
`df.transpose()` · `df.T`	Intercambia filas y columnas.
`df[[«c», «a», «b»]]`	Reordenar columnas: seleccionarlas en el orden deseado.

Reemplazar y transformar valores

Elemento	Qué hace
`df.replace(to_replace=None, value, inplace=False, regex=False)`	Reemplaza valores por otros: un valor, una lista o un diccionario {viejo: nuevo}. regex=True interpreta los patrones como expresiones regulares.
`df.clip(lower=None, upper=None, axis=None, inplace=False)`	Limita los valores a un rango: los menores a lower pasan a valer lower y los mayores a upper pasan a valer upper.
`df.round(decimals=0)`	Redondea; decimals puede ser un entero o un diccionario por columna.
`df.astype(dtype, copy, errors=«raise»)`	Convierte el tipo de una o varias columnas (sección 9).
`df.combine(other, func, fill_value=None, overwrite=True)`	Combina dos DataFrames columna por columna con una función que elijas.
`df.combine_first(other)`	Rellena los faltantes con los valores de otro DataFrame, alineando por etiquetas.

9. Tipos de datos (dtypes) y conversión

Cada columna tiene un tipo. Elegirlo bien ahorra memoria y evita errores en operaciones y comparaciones.

Principales tipos

Elemento	Qué hace
`int8 · int16 · int32 · int64 (uint8 ... uint64)`	Enteros con signo y sin signo de distinto tamaño. Si la columna tiene faltantes pasa a float64, salvo que se use un tipo que admita nulos.
`float32 · float64`	Números con decimales.
`bool`	Verdadero o falso; no admite faltantes.
`str`	Tipo de texto por defecto en pandas 3 (antes las columnas de texto eran de tipo object). El faltante sigue siendo NaN.
`object`	Tipo genérico de Python: queda para columnas con valores mezclados u objetos arbitrarios.
`string (pd.StringDtype)`	Tipo de texto explícito, que usa pd.NA como faltante. Admite storage «python» o «pyarrow».
`category (pd.CategoricalDtype(categories, ordered))`	Valores de un conjunto fijo de categorías, guardados como códigos. Ahorra memoria y permite orden lógico.
`datetime64[s | ms | us | ns]`	Fechas y horas. En pandas 3 la unidad se infiere de los datos (por ejemplo, microsegundos al leer texto) y no siempre es nanosegundos.
`timedelta64[unit]`	Duraciones.
`period[freq] · interval`	Períodos de tiempo (un mes, un trimestre) e intervalos numéricos o de fechas.
`Int8 · Int16 · Int32 · Int64 · UInt8 ... · Float32 · Float64 · boolean`	Tipos «anulables» (con mayúscula inicial): admiten pd.NA sin convertirse a flotante.
`pd.ArrowDtype (por ejemplo «int64[pyarrow]»)`	Tipos respaldados por Apache Arrow: más eficientes en memoria y con soporte de nulos.
`pd.SparseDtype`	Tipo para datos dispersos (mayoría de valores iguales).

Convertir tipos

Elemento	Qué hace
`df.astype(dtype, copy, errors=«raise»)`	Convierte a otro tipo. dtype puede ser un tipo para todo el objeto o un diccionario {columna: tipo}. errors=«ignore» deja la columna sin cambios si falla.
`df.convert_dtypes(infer_objects=True, convert_string=True, convert_integer=True, convert_boolean=True, convert_floating=True, dtype_backend=«numpy_nullable»)`	Elige automáticamente los tipos más adecuados con soporte de nulos (Int64, boolean, string...).
`df.infer_objects()`	Intenta convertir columnas object a un tipo más específico.
`pd.to_numeric(...) · pd.to_datetime(...) · pd.to_timedelta(...)`	Conversión tolerante a números, fechas y duraciones (admite errors=«coerce»).
`s.astype(«category»)`	Convierte a categoría.
`df.select_dtypes(include, exclude)`	Elige columnas según su tipo.

Verificación de tipos (pd.api.types)

Elemento	Qué hace
`is_numeric_dtype · is_any_real_numeric_dtype · is_integer_dtype · is_float_dtype · is_bool_dtype`	Indican si un tipo o arreglo es numérico, entero, flotante o booleano.
`is_string_dtype · is_object_dtype · is_categorical_dtype`	Indican si es de texto, de tipo object o categórico.
`is_datetime64_any_dtype · is_datetime64_dtype · is_timedelta64_dtype · is_period_dtype · is_interval_dtype`	Indican si es de fechas, duraciones, períodos o intervalos.
`is_scalar · is_list_like · is_dict_like · is_number · is_bool · is_float · is_integer · is_re`	Verifican el tipo de un valor suelto.
`infer_dtype(value, skipna=True)`	Describe en texto el tipo que contiene un arreglo («integer», «string», «mixed»...).
`pandas_dtype(dtype)`	Convierte un texto o tipo en el objeto dtype de pandas.
`union_categoricals(to_union, sort_categories, ignore_order)`	Une varios arreglos categóricos con categorías distintas.

10. Valores faltantes

pandas marca los datos ausentes como NaN, NA o NaT. Casi todas las estadísticas los ignoran por defecto (skipna=True).

Elemento	Qué hace
`df.isna() · df.isnull() · df.notna() · df.notnull()`	Máscara True/False de faltantes (o de presentes). df.isna().sum() cuenta los faltantes por columna.
`df.dropna(axis=0, how=«any», thresh=None, subset=None, inplace=False, ignore_index=False)`	Elimina filas (o columnas) con faltantes. how=«any» si hay alguno, «all» si son todos; thresh es la cantidad mínima de valores no nulos que debe tener; subset limita las columnas a mirar.
`df.fillna(value, axis=None, inplace=False, limit=None)`	Rellena faltantes con un valor, o con un diccionario {columna: valor}. Para rellenar con el valor anterior o siguiente hay que usar ffill y bfill.
`df.ffill(axis=None, inplace=False, limit=None, limit_area=None)`	Rellena cada faltante con el último valor válido anterior (forward fill).
`df.bfill(axis=None, inplace=False, limit=None, limit_area=None)`	Rellena con el siguiente valor válido (backward fill).
`df.interpolate(method=«linear», axis=0, limit=None, inplace=False, limit_direction=None, limit_area=None, **kwargs)`	Estima los faltantes por interpolación: «linear», «time», «index», «polynomial», «spline», «nearest», entre otros (algunos requieren scipy).
`df.replace(to_replace, value, ...)`	También sirve para convertir marcadores de texto («-», «S/D») en NaN o viceversa.
`df.combine_first(other)`	Rellena faltantes con valores de otro DataFrame.
`df.where(cond, other) · df.mask(cond, other)`	Reemplazo condicional de valores.
`df.count() · df.first_valid_index() · df.last_valid_index()`	Cantidad de valores presentes y posición del primero y último valor válido.

Elemento	Qué hace
`s.hasnans`	True si la Series contiene faltantes.
`pd.isna(x) · pd.notna(x)`	Versión función; también sirve con escalares.

Un NaN nunca es igual a otro NaN (np.nan == np.nan es False). Para detectar faltantes usá siempre isna(), nunca una comparación con ==.

11. Duplicados, orden y ranking

Eliminar repetidos y ordenar los datos.

Elemento	Qué hace
`df.duplicated(subset=None, keep=«first»)`	Serie booleana que marca filas repetidas. keep: «first» (marca desde la segunda aparición), «last» o False (marca todas las repetidas).
`df.drop_duplicates(subset=None, keep=«first», inplace=False, ignore_index=False)`	Elimina filas repetidas. subset limita las columnas que se comparan.
`df.sort_values(by, axis=0, ascending=True, inplace=False, kind=«quicksort», na_position=«last», ignore_index=False, key=None)`	Ordena por los valores de una o varias columnas. ascending acepta una lista (uno por columna); na_position decide dónde van los faltantes; key aplica una función antes de comparar (p. ej. str.lower).
`df.sort_index(axis=0, level=None, ascending=True, inplace=False, kind, na_position, sort_remaining, ignore_index, key)`	Ordena por las etiquetas del índice (o de las columnas con axis=1).
`df.rank(axis=0, method=«average», numeric_only=False, na_option=«keep», ascending=True, pct=False)`	Posición de cada valor en el orden. method: «average», «min», «max», «first», «dense» (cómo se resuelven los empates); pct=True da el percentil.
`df.nlargest(n, columns, keep) · df.nsmallest(n, columns, keep)`	Los n mayores o menores valores, sin ordenar todo el DataFrame.
`s.argsort() · s.argmax() · s.argmin()`	Posiciones que ordenarían la Series y posición del máximo y del mínimo.
`s.searchsorted(value, side=«left», sorter=None)`	Posición donde insertar un valor para mantener el orden (la Series debe estar ordenada).
`s.is_monotonic_increasing · s.is_monotonic_decreasing`	Verifican si ya está ordenada.

12. Aritmética, comparación y lógica

Las operaciones entre objetos se alinean por etiquetas: si una etiqueta existe en un solo lado, el resultado es NaN (salvo que se indique fill_value). Los operadores +, -, *, /, //, %, ** funcionan columna a columna.

Operaciones

Elemento	Qué hace
`df.add(other, axis=«columns», level=None, fill_value=None) · df.radd(...)`	Suma (operador +). La versión con r invierte los operandos. axis permite operar contra una Series por filas.
`df.sub(...) · df.subtract(...) · df.rsub(...)`	Resta (operador -).
`df.mul(...) · df.multiply(...) · df.rmul(...)`	Multiplicación (operador *).
`df.div(...) · df.divide(...) · df.truediv(...) · df.rdiv(...) · df.rtruediv(...)`	División (operador /).
`df.floordiv(...) · df.rfloordiv(...)`	División entera (operador //).
`df.mod(...) · df.rmod(...)`	Resto (operador %).
`df.pow(...) · df.rpow(...)`	Potencia (operador **).
`Series.divmod(other) · Series.rdivmod(other)`	Cociente entero y resto en una sola operación.
`df.dot(other)`	Producto matricial con otro DataFrame, Series o arreglo.
`df.abs()`	Valor absoluto.
`df.round(decimals) · df.clip(lower, upper)`	Redondeo y limitación de rango.

Comparación y lógica

Elemento	Qué hace
`df.eq(other) · df.ne(other)`	Igual y distinto (operadores == y !=).
`df.lt(other) · df.le(other) · df.gt(other) · df.ge(other)`	Menor, menor o igual, mayor y mayor o igual (operadores <, <=, >, >=).
`& (y) · | (o) · ~ (no) · ^ (o exclusivo)`	Operadores lógicos entre máscaras booleanas. Cada condición va entre paréntesis.
`df.all(axis=0, bool_only, skipna) · df.any(axis=0, bool_only, skipna)`	True si todos o si algún valor de cada columna (o fila) es verdadero.
`df.equals(other)`	Igualdad completa entre dos objetos.
`df.compare(other, align_axis=1, keep_shape=False, keep_equal=False, result_names=(«self», «other»))`	Muestra solo las diferencias entre dos DataFrames con las mismas etiquetas.

Cambios acumulados y entre filas

Elemento	Qué hace
`df.diff(periods=1, axis=0)`	Diferencia con la fila anterior (o la de periods filas atrás).
`df.pct_change(periods=1, fill_method=None, freq=None)`	Variación porcentual respecto de la fila anterior (0,10 equivale a +10 %).
`df.shift(periods=1, freq=None, axis=0, fill_value, suffix)`	Desplaza los valores hacia abajo (o arriba con negativos); útil para comparar con el período anterior.
`df.cumsum() · df.cumprod() · df.cummax() · df.cummin()`	Suma, producto, máximo y mínimo acumulados.

13. Estadísticas y agregaciones

Resumen de columnas (axis=0, por defecto) o de filas (axis=1). skipna=True ignora los faltantes; numeric_only=True limita el cálculo a columnas numéricas (con False, una columna de texto puede dar error).

Elemento	Qué hace
`df.sum(axis=0, skipna=True, numeric_only=False, min_count=0)`	Suma. min_count es la cantidad mínima de valores válidos para dar resultado (si no, NaN).
`df.mean(...) · df.median(...)`	Media y mediana.
`df.mode(axis=0, numeric_only=False, dropna=True)`	Valor o valores más frecuentes (puede devolver varios).
`df.min(...) · df.max(...)`	Mínimo y máximo.
`df.prod(...) · df.product(...)`	Producto de los valores.
`df.std(axis, skipna, ddof=1, numeric_only) · df.var(...)`	Desvío estándar y varianza. ddof=1 usa n-1 (muestral); ddof=0 usa n (poblacional).
`df.sem(...)`	Error estándar de la media.
`df.skew(...) · df.kurt(...) · df.kurtosis(...)`	Asimetría y curtosis.
`df.quantile(q=0.5, axis=0, numeric_only=False, interpolation=«linear», method=«single»)`	Cuantil o lista de cuantiles (0,25, 0,75...). interpolation: «linear», «lower», «higher», «midpoint», «nearest».
`df.count() · df.nunique() · df.size`	Cantidad de valores no nulos, de valores distintos y de elementos totales.
`df.value_counts(subset, normalize, sort, ascending, dropna)`	Frecuencia de cada valor o combinación.
`df.describe(percentiles, include, exclude)`	Resumen estadístico completo.
`df.corr(method=«pearson», min_periods=1, numeric_only=False)`	Matriz de correlación entre columnas. method: «pearson», «kendall» o «spearman».
`df.cov(min_periods=None, ddof=1, numeric_only=False)`	Matriz de covarianza.
`df.corrwith(other, axis=0, drop=False, method, numeric_only)`	Correlación de cada columna con una Series u otro DataFrame.
`df.idxmax(...) · df.idxmin(...)`	Etiqueta del máximo y del mínimo.
`df.agg(func=None, axis=0, *args, **kwargs) · df.aggregate(...)`	Aplica una o varias funciones de resumen: un nombre («sum»), una lista ([«min», «max»]) o un diccionario {columna: función}.
`df.all() · df.any()`	Verifican si todos o algunos valores son verdaderos.
`df.cumsum() · cumprod() · cummax() · cummin()`	Acumulados (sección 12).
`s.autocorr(lag=1)`	Autocorrelación de una Series con ella misma desplazada lag posiciones.
`df.memory_usage(index, deep)`	Memoria utilizada por columna.

14. Aplicar funciones

Cuando ninguna operación vectorizada alcanza, se pueden aplicar funciones propias. Son más lentas que las operaciones vectorizadas (df[«a»] + df[«b»]), así que conviene usarlas como último recurso.

Elemento	Qué hace
`df.apply(func, axis=0, raw=False, result_type=None, args=(), by_row=«compat», engine=None, engine_kwargs=None, **kwargs)`	Aplica una función a cada columna (axis=0) o a cada fila (axis=1). La función recibe una Series. raw=True le pasa un arreglo de NumPy; result_type controla la forma del resultado («expand», «reduce», «broadcast»).
`df.map(func, na_action=None, **kwargs)`	Aplica una función a cada elemento individual del DataFrame. Reemplaza al antiguo applymap, que se eliminó en pandas 3.
`Series.map(arg, na_action=None)`	Transforma cada valor con una función, un diccionario o una Series de equivalencias. Lo que no está en el diccionario pasa a NaN.
`Series.apply(func, args=(), by_row=«compat», **kwargs)`	Aplica una función a cada valor de la Series.
`df.agg(func) · df.aggregate(func)`	Aplica funciones que reducen (resumen) cada columna.
`df.transform(func, axis=0, *args, **kwargs)`	Aplica una función que devuelve un resultado de la misma forma que la entrada (útil con groupby).
`df.pipe(func, *args, **kwargs)`	Pasa el DataFrame como primer argumento de una función; permite encadenar pasos con funciones propias.
`Series.case_when(caselist)`	Asigna valores según una lista de pares (condición, valor), al estilo CASE WHEN de SQL.
`df.eval(expr, inplace=False, **kwargs)`	Evalúa una expresión de texto con columnas, p. ej. «c = a + b», y puede crear columnas nuevas.
`df.iterrows()`	Recorre filas como pares (índice, Series). Lento; conviene evitarlo.
`df.itertuples(index=True, name=«Pandas»)`	Recorre filas como tuplas con nombre. Más rápido que iterrows.
`df.items()`	Recorre columnas como pares (nombre, Series).
`np.where(cond, a, b) · np.select(condiciones, valores, default)`	Alternativas vectorizadas de NumPy para columnas condicionales.

15. Agrupar datos (groupby)

groupby divide la tabla en grupos según una o más columnas, aplica una operación a cada grupo y junta los resultados.

Crear el agrupamiento

Elemento	Qué hace
`df.groupby(by=None, level=None, as_index=True, sort=True, group_keys=True, observed=True, dropna=True)`	Devuelve un objeto DataFrameGroupBy. by es una columna, lista de columnas, función o Series. as_index=False deja las claves como columnas normales; sort ordena los grupos por clave; dropna=False conserva el grupo de claves faltantes; observed controla si se muestran categorías sin datos.
`for nombre, grupo in df.groupby(...)`	Iterar entrega cada clave y el DataFrame de ese grupo.
`gb[«col»] · gb[[«a», «b»]]`	Elegir columnas sobre las que operar, antes de agregar.
`gb.get_group(name)`	Devuelve el DataFrame de un grupo concreto.
`gb.groups · gb.indices`	Diccionarios con las etiquetas o las posiciones de las filas de cada grupo.
`gb.ngroups · gb.ngroup()`	Cantidad de grupos y número de grupo al que pertenece cada fila.
`pd.Grouper(key, level, freq, sort, dropna)`	Agrupa por columna, por nivel del índice o por frecuencia de tiempo (p. ej. freq=«MS» para mes).

Agregaciones (devuelven un valor por grupo)

Elemento	Qué hace
`gb.agg(func) · gb.aggregate(func)`	Agregación flexible: un nombre («sum»), una lista, un diccionario {columna: función} o agregación con nombre: agg(total=(«monto», «sum»), media=(«monto», «mean»)), que también se escribe con pd.NamedAgg.
`gb.sum() · gb.mean() · gb.median() · gb.min() · gb.max() · gb.prod()`	Suma, media, mediana, mínimo, máximo y producto de cada grupo.
`gb.std() · gb.var() · gb.sem() · gb.skew() · gb.kurt()`	Desvío, varianza, error estándar, asimetría y curtosis.
`gb.count() · gb.size() · gb.nunique()`	Cantidad de valores no nulos, cantidad de filas (incluidos faltantes) y cantidad de valores distintos.
`gb.first() · gb.last() · gb.nth(n)`	Primer valor no nulo, último, y la fila en la posición n de cada grupo.
`gb.quantile(q=0.5)`	Cuantil de cada grupo.
`gb.idxmax() · gb.idxmin()`	Etiqueta del máximo o mínimo en cada grupo.
`gb.any() · gb.all()`	Si algún valor o todos son verdaderos.
`gb.value_counts() · gb.describe()`	Frecuencias y resumen estadístico por grupo.
`gb.ohlc()`	Apertura, máximo, mínimo y cierre de cada grupo (datos financieros).
`gb.corr() · gb.cov() · gb.corrwith(other)`	Correlación y covarianza dentro de cada grupo.

Transformaciones y filtros (conservan filas)

Elemento	Qué hace
`gb.transform(func)`	Devuelve un resultado con el mismo índice que el original (por ejemplo, el total del grupo repetido en cada fila). Sirve para porcentajes sobre el grupo o para rellenar con la media del grupo.
`gb.apply(func, ...)`	Aplica una función arbitraria a cada grupo (recibe un DataFrame) y junta los resultados.
`gb.filter(func, dropna=True)`	Conserva solo los grupos para los que la función devuelve True (p. ej. grupos con más de 5 filas).
`gb.head(n) · gb.tail(n) · gb.sample(n, frac, ...) · gb.take(indices)`	Primeras, últimas filas, muestra y selección por posición dentro de cada grupo.
`gb.cumsum() · gb.cumprod() · gb.cummax() · gb.cummin() · gb.cumcount()`	Acumulados dentro de cada grupo; cumcount numera las filas de cada grupo desde 0.
`gb.rank(...) · gb.shift(periods) · gb.diff() · gb.pct_change()`	Ranking, desplazamiento, diferencia y variación porcentual por grupo.
`gb.ffill() · gb.bfill()`	Rellena faltantes dentro de cada grupo.
`gb.rolling(...) · gb.expanding(...) · gb.ewm(...) · gb.resample(rule)`	Ventanas móviles y remuestreo temporal dentro de cada grupo.
`gb.pipe(func) · gb.plot · gb.hist() · gb.boxplot()`	Encadenar funciones y graficar por grupo.

16. Combinar tablas

Unir DataFrames por columnas clave (como un JOIN de SQL) o apilarlos.

Elemento	Qué hace
`pd.merge(left, right, how=«inner», on=None, left_on=None, right_on=None, left_index=False, right_index=False, sort=False, suffixes=(«_x», «_y»), indicator=False, validate=None)` · `df.merge(right, ...)`	Une dos tablas por una o más columnas. how: «inner» (solo coincidencias), «left», «right», «outer» (todo) o «cross» (producto cartesiano). on es la clave común; left_on/right_on son claves con nombres distintos; left_index/right_index usan el índice. suffixes distingue columnas repetidas.
`merge(..., indicator=True)`	Agrega una columna «_merge» que dice si cada fila vino de la izquierda, la derecha o ambas (útil para auditar uniones).
`merge(..., validate=«one_to_one»)`	Verifica la relación esperada («one_to_one» o «1:1», «one_to_many», «many_to_one», «many_to_many») y da error si se incumple; evita multiplicar filas por claves repetidas sin darse cuenta.
`df.join(other, on=None, how=«left», lsuffix=«», rsuffix=«», sort=False, validate=None)`	Une por índice (o por una columna de df contra el índice de other). Es un atajo de merge, con left por defecto.
`pd.concat(objs, axis=0, join=«outer», ignore_index=False, keys=None, levels=None, names=None, verify_integrity=False, sort=False)`	Apila objetos: axis=0 uno debajo del otro, axis=1 lado a lado. join=«inner» conserva solo columnas comunes. ignore_index=True renumera el índice; keys agrega un nivel con el origen de cada pieza. Es el reemplazo del antiguo df.append.
`pd.merge_ordered(left, right, on, left_on, right_on, left_by, right_by, fill_method, suffixes, how=«outer»)`	Unión pensada para datos ordenados (series de tiempo), con relleno opcional.
`pd.merge_asof(left, right, on, left_on, right_on, left_index, right_index, by, left_by, right_by, suffixes, tolerance, allow_exact_matches, direction=«backward»)`	Une cada fila con la más cercana (no necesariamente igual) de la otra tabla, por ejemplo la cotización anterior más próxima en el tiempo. Ambas tablas deben estar ordenadas por la clave.
`df.combine_first(other)`	Completa faltantes con otro DataFrame.
`df.update(other)`	Sobrescribe valores con los de otro DataFrame, alineando por etiquetas.
`df.compare(other)` · `df.align(other)`	Diferencias entre dos DataFrames y alineación por etiquetas.

Cuidado: si la clave está repetida en ambas tablas, merge multiplica las filas (3 coincidencias por 2 coincidencias dan 6 filas). Usá validate para detectarlo.

17. Reorganizar y transformar la forma de los datos

Pasar de formato «ancho» (una columna por categoría) a «largo» (una fila por observación) y viceversa, o resumir en tablas cruzadas.

Elemento	Qué hace
`df.pivot(columns, index=None, values=None)` · `pd.pivot(data, ...)`	Reorganiza de largo a ancho: los valores únicos de columns pasan a ser columnas. Falla si hay combinaciones índice-columna repetidas.
`df.pivot_table(values=None, index=None, columns=None, aggfunc=«mean», fill_value=None, margins=False, dropna=True, margins_name=«All», observed=True, sort=True)`	Tabla dinámica: como pivot pero agregando los repetidos con aggfunc («sum», «count», lista de funciones, diccionario). margins=True agrega totales; fill_value reemplaza los vacíos.

Elemento	Qué hace
`df.melt(id_vars=None, value_vars=None, var_name=None, value_name=«value», col_level=None, ignore_index=True)`	De ancho a largo: deja id_vars fijas y convierte el resto de las columnas en pares (variable, valor).
`pd.wide_to_long(df, stubnames, i, j, sep=«», suffix=«\d+»)`	Versión de melt para columnas con un prefijo común y un sufijo numérico (ventas2024, ventas2025).
`pd.lreshape(data, groups, dropna=True)`	Pasa de ancho a largo agrupando varias columnas en una.
`df.stack(level=-1, dropna, sort, future_stack=True)`	Lleva columnas al índice (de ancho a largo), produciendo un MultiIndex. En pandas 3 usa la implementación nueva.
`df.unstack(level=-1, fill_value=None, sort=True)`	Lleva un nivel del índice a columnas.
`df.explode(column, ignore_index=False)`	Una fila por cada elemento de una lista en la columna.
`pd.crosstab(index, columns, values=None, rownames=None, colnames=None, aggfunc=None, margins=False, margins_name=«All», dropna=True, normalize=False)`	Tabla de frecuencias cruzadas entre dos o más variables. normalize pasa a proporciones («index», «columns», «all»).
`pd.get_dummies(data, prefix=None, prefix_sep=«_», dummy_na=False, columns=None, sparse=False, drop_first=False, dtype=None)`	Codificación one-hot: una columna 0/1 (o True/False) por cada categoría.
`pd.from_dummies(data, sep=None, default_category=None)`	Operación inversa a get_dummies.
`pd.cut(x, bins, right=True, labels=None, retbins=False, precision=3, include_lowest=False, duplicates=«raise», ordered=True)`	Agrupa valores continuos en intervalos definidos por vos (bins) y devuelve una categoría por valor.
`pd.qcut(x, q, labels=None, retbins=False, precision=3, duplicates=«raise»)`	Agrupa por cuantiles: cada tramo contiene aproximadamente la misma cantidad de datos.
`df.transpose() · df.T`	Intercambia filas y columnas.
`pd.factorize(values) · Series.factorize()`	Codifica valores como números enteros.

18. Series: métodos propios y accesores

Además de casi todo lo anterior, la Series tiene métodos que no existen en el DataFrame.

Elemento	Qué hace
`s.to_frame(name)`	Convierte la Series en un DataFrame de una columna.
`s.to_list() · s.tolist()`	Convierte a una lista de Python.
`s.unique()`	Valores únicos en orden de aparición (devuelve un arreglo).
`s.nunique(dropna=True)`	Cantidad de valores distintos.
`s.value_counts(normalize, sort, ascending, bins, dropna)`	Frecuencia de cada valor; bins agrupa en intervalos.
`s.factorize()`	Codifica los valores como enteros y devuelve (códigos, únicos).
`s.between(left, right, inclusive=«both»)`	True entre dos límites.

Elemento	Qué hace
`s.isin(values)`	True si el valor está en una lista.
`s.map(arg, na_action)` · `s.apply(func)`	Transforman cada valor con una función, un diccionario o una Series.
`s.case_when(caselist)`	Asignación condicional con varias condiciones.
`s.argmax()` · `s.argmin()` · `s.argsort()`	Posición del máximo, del mínimo y orden de los valores.
`s.searchsorted(value, side, sorter)`	Posición de inserción en una Series ordenada.
`s.repeat(repeats)`	Repite cada elemento esa cantidad de veces.
`s.autocorr(lag=1)`	Autocorrelación con retardo lag.
`s.item()`	Devuelve el único elemento de una Series de largo 1.
`s.divmod(other)`	Cociente y resto.
`s.update(other)`	Actualiza valores con los de otra Series, alineando por etiqueta.
`s.compare(other)`	Diferencias entre dos Series.
`s.str` · `s.dt` · `s.cat`	Accesores de texto, fecha y categoría (secciones 19 a 21).
`s.list` · `s.struct`	Accesores para columnas de tipo lista o estructura (tipos Arrow): permiten extraer elementos o campos anidados.

19. Texto: accesor .str

Métodos vectorizados de texto que se aplican a toda una columna (s.str.metodo). Los faltantes se propagan sin dar error. Solo funcionan sobre columnas de texto.

Búsqueda y comprobación

Elemento	Qué hace
`s.str.contains(pat, case=True, flags=0, na, regex=True)`	True si el texto contiene el patrón. regex=False busca el texto literal; case=False ignora mayúsculas.
`s.str.startswith(pat, na)` · `s.str.endswith(pat, na)`	True si empieza o termina con el texto dado.
`s.str.match(pat, case, flags, na)` · `s.str.fullmatch(pat, ...)`	True si el patrón coincide desde el inicio (match) o con todo el texto (fullmatch).
`s.str.find(sub, start, end)` · `s.str.rfind(...)`	Posición de la primera o de la última aparición (-1 si no está).
`s.str.index(sub)` · `s.str.rindex(sub)`	Como find, pero da error si no encuentra.
`s.str.count(pat, flags)`	Cantidad de veces que aparece un patrón.
`s.str.len()`	Largo de cada texto.
`s.str.isalnum()` · `isalpha()` · `isdigit()` · `isdecimal()` · `isnumeric()` · `isspace()` · `islower()` · `isupper()` · `istitle()` · `isascii()`	Comprobaciones del tipo de caracteres que contiene cada texto.

Extraer y dividir

Elemento	Qué hace
`s.str.extract(pat, flags, expand=True)`	Extrae el primer grupo de captura de la expresión regular de cada texto.
`s.str.extractall(pat, flags)`	Extrae todas las coincidencias, una fila por cada una.
`s.str.findall(pat, flags)`	Lista de todas las coincidencias del patrón.
`s.str.split(pat=None, n=-1, expand=False, regex=None)`	Divide en partes. expand=True devuelve cada parte en su propia columna; n limita la cantidad de cortes.
`s.str.rsplit(pat, n, expand)`	Como split pero contando desde la derecha.
`s.str.partition(sep, expand)` · `s.str.rpartition(sep, expand)`	Divide en tres partes: antes, separador y después (desde izquierda o derecha).
`s.str.get(i)`	Elemento en la posición i de cada texto o lista.
`s.str.slice(start, stop, step)`	Recorta una parte del texto por posición.
`s.str.get_dummies(sep=«|»)`	Columnas 0/1 a partir de textos con varios valores separados por sep.

Modificar

Elemento	Qué hace
`s.str.replace(pat, repl, n=-1, case=None, flags=0, regex=False)`	Reemplaza texto. Con regex=False (por defecto) el patrón es literal; con regex=True es una expresión regular y repl puede ser una función.
`s.str.slice_replace(start, stop, repl)`	Reemplaza una porción por posición.
`s.str.strip(to_strip)` · `s.str.lstrip(to_strip)` · `s.str.rstrip(to_strip)`	Quita espacios (o los caracteres dados) de ambos lados, del izquierdo o del derecho.
`s.str.removeprefix(prefix)` · `s.str.removesuffix(suffix)`	Quita un prefijo o un sufijo si está presente.
`s.str.lower()` · `upper()` · `title()` · `capitalize()` · `swapcase()` · `casefold()`	Cambian mayúsculas y minúsculas (casefold es la versión agresiva para comparar).
`s.str.pad(width, side, fillchar)` · `center(width, fillchar)` · `ljust(width, fillchar)` · `rjust(width, fillchar)` · `zfill(width)`	Rellenan hasta un ancho dado; zfill agrega ceros a la izquierda (útil para códigos).
`s.str.cat(others=None, sep=None, na_rep=None, join=«left»)`	Concatena los textos de la Series (o con otra Series, elemento a elemento).
`s.str.join(sep)`	Une los elementos de cada lista con un separador.
`s.str.repeat(repeats)`	Repite cada texto.
`s.str.wrap(width)`	Inserta saltos de línea para ajustar el ancho.
`s.str.normalize(form)`	Normaliza caracteres Unicode («NFC», «NFD», «NFKC», «NFKD»); sirve, por ejemplo, para quitar tildes tras descomponerlos.
`s.str.translate(table)`	Reemplaza caracteres según una tabla de equivalencias.
`s.str.encode(encoding)` · `s.str.decode(encoding)`	Convierte entre texto y bytes.

20. Fechas: accesor .dt

Extrae partes de columnas de fecha-hora (s.dt.atributo). La columna debe ser de tipo datetime; si es texto, antes hay que convertirla con pd.to_datetime.

Componentes de fecha y hora

Elemento	Qué hace
`s.dt.year · month · day · hour · minute · second · microsecond · nanosecond`	Cada componente como número.
`s.dt.dayofweek · s.dt.weekday · s.dt.day_of_week`	Día de la semana: lunes es 0 y domingo es 6.
`s.dt.dayofyear · s.dt.day_of_year`	Día del año (1 a 366).
`s.dt.quarter`	Trimestre (1 a 4).
`s.dt.days_in_month · s.dt.daysinmonth`	Cantidad de días del mes.
`s.dt.isocalendar()`	DataFrame con año, semana y día según el calendario ISO (para la semana: s.dt.isocalendar().week).
`s.dt.date · s.dt.time · s.dt.timetz`	Solo la fecha o solo la hora (como objetos de Python).
`s.dt.is_month_start · is_month_end · is_quarter_start · is_quarter_end · is_year_start · is_year_end · is_leap_year`	Indicadores booleanos.
`s.dt.day_name(locale=None) · s.dt.month_name(locale=None)`	Nombre del día o del mes, en texto. locale=«es_ES» lo da en español si el sistema lo tiene.

Formato y redondeo

Elemento	Qué hace
`s.dt.strftime(format)`	Convierte a texto con un formato, p. ej. «%d/%m/%Y» (día/mes/año) o «%Y-%m».
`s.dt.normalize()`	Lleva la hora a medianoche (deja solo la fecha, aún como datetime).
`s.dt.floor(freq) · s.dt.ceil(freq) · s.dt.round(freq)`	Redondea hacia abajo, hacia arriba o al más cercano a una frecuencia («h», «D», «15min»...).
`s.dt.as_unit(unit) · s.dt.unit`	Cambia o consulta la resolución interna («s», «ms», «us», «ns»).
`s.dt.to_pydatetime()`	Convierte a objetos datetime de Python.

Zonas horarias y períodos

Elemento	Qué hace
`s.dt.tz · s.dt.tz_localize(tz, ambiguous, nonexistent)`	tz indica la zona horaria; tz_localize asigna una zona a fechas «ingenuas» (sin zona).
`s.dt.tz_convert(tz)`	Convierte de una zona horaria a otra, p. ej. «America/Argentina/Buenos_Aires».
`s.dt.to_period(freq)`	Convierte a períodos («M» para mes, «Q» trimestre, «Y» año).
`s.dt.start_time · s.dt.end_time · s.dt.qyear · s.dt.asfreq(freq) · s.dt.freq`	Para columnas de períodos: inicio y fin del período, año de trimestre y cambio de frecuencia.

Duraciones (columnas timedelta)

Elemento	Qué hace
`s.dt.days · seconds · microseconds · nanoseconds`	Componentes de la duración.
`s.dt.total_seconds()`	Duración total expresada en segundos (para horas: dividir por 3600; para días, por 86400).
`s.dt.components`	DataFrame con días, horas, minutos, segundos, etc.
`s.dt.to_pytimedelta()`	Convierte a timedelta de Python.

21. Categorías: accesor .cat

Para columnas de tipo category (s.cat.metodo).

Elemento	Qué hace
`s.cat.categories`	Índice con las categorías.
`s.cat.ordered`	True si las categorías tienen un orden lógico.
`s.cat.codes`	Código numérico de cada valor (-1 indica faltante).
`s.cat.add_categories(new_categories)`	Agrega categorías nuevas (aún sin datos).
`s.cat.remove_categories(removals)`	Quita categorías; los valores afectados pasan a faltantes.
`s.cat.remove_unused_categories()`	Quita las categorías que no aparecen en los datos.
`s.cat.rename_categories(new_categories)`	Cambia los nombres de las categorías (lista o diccionario).
`s.cat.reorder_categories(new_categories, ordered)`	Reordena las categorías existentes.
`s.cat.set_categories(new_categories, ordered, rename)`	Define el conjunto exacto de categorías: agrega y quita en un solo paso.
`s.cat.as_ordered() · s.cat.as_unordered()`	Marca la categoría como ordenada o desordenada (permite comparar y usar min/max).
`pd.CategoricalDtype(categories=None, ordered=False)`	Tipo categórico reutilizable en varias columnas, con astype.
`pd.api.types.union_categoricals(to_union)`	Une arreglos categóricos con categorías diferentes.

22. Fechas y series de tiempo

Conversión de fechas, rangos, remuestreo y zonas horarias. Con un índice de tipo fecha (DatetimeIndex) se puede cortar por texto, como df.loc[«2026-03»].

Conversión y rangos

Elemento	Qué hace
`pd.to_datetime(arg, errors=«raise», dayfirst=False, yearfirst=False, utc=False, format=None, exact, unit=None, origin=«unix», cache=True)`	Convierte texto, números o columnas a fecha. format indica el formato exacto (p. ej. «%d/%m/%Y»), lo que es más rápido y seguro; dayfirst=True interpreta el día primero; errors=«coerce» convierte los inválidos en NaT; unit y origin sirven para números (segundos desde 1970 o fechas de Excel).
`pd.date_range(start, end, periods, freq, tz, normalize, name, inclusive=«both», unit)`	Genera una secuencia de fechas entre dos extremos, o desde un inicio con cierta cantidad de períodos (periods) y frecuencia (freq).
`pd.bdate_range(start, end, periods, freq=«B», tz, normalize, name, weekmask, holidays, inclusive)`	Como date_range pero con días hábiles (puede indicarse feriados).
`pd.period_range(start, end, periods, freq)`	Secuencia de períodos (meses, trimestres, años).
`pd.timedelta_range(start, end, periods, freq)`	Secuencia de duraciones.
`pd.interval_range(start, end, periods, freq, closed)`	Secuencia de intervalos.
`pd.infer_freq(index)`	Deduce la frecuencia de un índice de fechas.
`pd.to_timedelta(arg, unit)`	Convierte a duración (p. ej. «3 days», 1.5 con unit=«h»).

Escalares de tiempo

Elemento	Qué hace
`pd.Timestamp(ts_input, tz=None, unit=None)`	Un instante. Tiene componentes (year, month, day, hour...), dayofweek, day_name(), month_name(), strftime(), isoformat(), normalize(), floor/ceil/round, tz_localize, tz_convert, to_period, to_pydatetime, timestamp, replace, y los atributos de clase Timestamp.now() y Timestamp.today().
`pd.Timedelta(value, unit)`	Una duración. Tiene days, seconds, total_seconds(), components, floor/ceil/round, to_pytimedelta(), isoformat().
`pd.Period(value, freq)`	Un período de tiempo (un mes, un año...). Tiene start_time, end_time, asfreq(freq), to_timestamp(), strftime(), year, month, quarter.
`pd.Interval(left, right, closed)`	Intervalo con left, right, mid, length, closed, overlaps(other), is_empty.
`pd.DateOffset(years, months, weeks, days, hours, minutes, seconds, ...)`	Desplazamiento de calendario, que respeta meses y años de distinto largo: Timestamp + pd.DateOffset(months=1).
`pd.offsets`	Módulo con desplazamientos específicos (MonthEnd, MonthBegin, BDay, QuarterEnd, YearEnd, Week...).

Operaciones temporales

Elemento	Qué hace
`df.resample(rule, closed=None, label=None, convention=«start», on=None, level=None, origin=«start_day», offset=None, group_keys=False)`	Cambia la frecuencia de una serie temporal (como un groupby por período). rule es la frecuencia («D», «W», «MS», «ME»...). on indica la columna de fechas si no está en el índice; closed y label definen de qué lado se cierra cada intervalo y cómo se etiqueta. Devuelve un Resampler al que se aplica una agregación.
`resampler.sum() · mean() · median() · min() · max() · std() · var() · sem() · prod() · count() · size() · nunique() · first() · last() · quantile() · ohlc()`	Agregaciones por intervalo.
`resampler.agg(func) · apply(func) · transform(func) · pipe(func)`	Agregación flexible y funciones propias.
`resampler.asfreq() · ffill() · bfill() · nearest() · interpolate()`	Para aumentar la frecuencia (pasar de mensual a diario): rellenan los nuevos huecos.
`resampler.groups · indices · ngroups · get_group(name)`	Información de los intervalos formados.
`df.asfreq(freq, method, how, normalize, fill_value)`	Cambia la frecuencia sin agregar: toma el valor en cada punto de la nueva grilla, y rellena si se indica.
`df.shift(periods, freq=None, axis, fill_value, suffix)`	Desplaza datos; con freq desplaza las etiquetas de tiempo en lugar de los datos.
`df.tz_localize(tz) · df.tz_convert(tz)`	Asigna o convierte la zona horaria del índice.
`df.to_period(freq) · df.to_timestamp()`	Convierte el índice entre fechas y períodos.
`df.at_time(time) · df.between_time(start, end) · df.truncate(before, after)`	Cortes por hora del día o por rango de fechas.
`df.first_valid_index() · df.last_valid_index()`	Primera y última fecha con datos.
`df.rolling(«7D»)`	Ventana móvil por tiempo (7 días); ver sección 23.

Frecuencias más usadas (freq / rule)

Elemento	Qué hace
`«D» · «B» · «W»`	Día calendario, día hábil y semana.
`«MS» · «ME»`	Inicio de mes y fin de mes.
`«QS» · «QE»`	Inicio y fin de trimestre.
`«YS» · «YE»`	Inicio y fin de año.
`«h» · «min» · «s» · «ms»`	Hora, minuto, segundo y milisegundo (en minúsculas).
`«15min» · «2W» · «3MS»`	Un número delante multiplica la frecuencia.

En pandas 3 los alias antiguos «M», «Q», «Y», «A», «H», «T» y «S» ya no se aceptan: se usan «ME», «QE», «YE», «h», «min» y «s». Esto rompe mucho código viejo de tutoriales.

23. Ventanas móviles (rolling, expanding, ewm)

Cálculos sobre ventanas deslizantes, acumuladas o ponderadas exponencialmente.

df.rolling(window, min_periods=None, center=False, win_type=None, on=None, closed=None, step=None, method=«single»)

Elemento	Qué hace
`window`	Tamaño de la ventana: cantidad de filas (entero) o un período de tiempo («7D») si el índice es de fechas. También puede ser un indexador propio (pd.api.indexers.BaseIndexer).
`min_periods`	Cantidad mínima de valores válidos en la ventana para dar resultado.
`center=True`	Centra la ventana en lugar de usar solo valores anteriores.
`win_type`	Ponderación de la ventana («gaussian», «triang», «hamming»...; requiere scipy).
`on · closed · step`	Columna de fechas a usar como eje, qué extremos incluye la ventana y paso entre ventanas.
`rolling.sum() · mean() · median() · min() · max() · std() · var() · sem() · skew() · kurt() · count() · nunique()`	Estadísticas sobre cada ventana.
`rolling.quantile(q) · rolling.rank() · rolling.first() · rolling.last()`	Cuantil, posición dentro de la ventana, primer y último valor.
`rolling.corr(other) · rolling.cov(other)`	Correlación y covarianza móviles con otra serie.
`rolling.apply(func, raw=False, engine, ...)`	Aplica una función propia a cada ventana.
`rolling.agg(func) · rolling.aggregate(func) · rolling.pipe(func)`	Varias agregaciones a la vez y encadenado.

df.expanding(min_periods=1, method=«single»)

Elemento	Qué hace
`expanding.sum() · mean() · median() · min() · max() · std() · var() · count() ...`	Igual que rolling, pero la ventana crece desde el inicio hasta la fila actual (promedio o suma acumulados).
`expanding.apply(func) · agg(func) · quantile(q) · corr(other) · cov(other)`	Función propia, agregación flexible, cuantil y correlación.

df.ewm(com, span, halflife, alpha, min_periods=0, adjust=True, ignore_na=False, times=None, method=«single»)

Elemento	Qué hace
`com · span · halflife · alpha`	Formas equivalentes de indicar cuánto pesa el pasado en el promedio exponencial; se elige solo una.
`adjust · ignore_na · times`	Corrección de los pesos al inicio, tratamiento de faltantes y tiempos reales entre observaciones.
`ewm.mean() · std() · var() · sum() · corr(other) · cov(other) · online()`	Estadísticas exponenciales; online permite actualizar con datos nuevos.

Indexadores propios (pd.api.indexers)

Elemento	Qué hace
`BaseIndexer` · `FixedForwardWindowIndexer` · `VariableOffsetWindowIndexer`	Clases para definir ventanas a medida: hacia adelante, de tamaño variable por fecha, etc.

24. Index y MultiIndex

El índice etiqueta las filas (y las columnas son otro Index). Permite alinear datos y hacer búsquedas rápidas.

Index (comunes a todos los tipos de índice)

Elemento	Qué hace
`idx.name · idx.names · idx.set_names(names) · idx.rename(name)`	Nombre del índice (o de cada nivel) y formas de cambiarlo.
`idx.values · idx.to_numpy() · idx.to_list() · idx.tolist() · idx.to_series() · idx.to_frame()`	Convierten el índice a arreglo, lista, Series o DataFrame.
`idx.shape · idx.size · idx.ndim · idx.nbytes · idx.dtype · idx.inferred_type · idx.empty`	Forma, tamaño, memoria y tipo.
`idx.is_unique · idx.has_duplicates · idx.is_monotonic_increasing · idx.is_monotonic_decreasing`	Propiedades de unicidad y de orden.
`idx.get_loc(key) · idx.get_indexer(target) · idx.get_indexer_for(target) · idx.get_indexer_non_unique(target)`	Posición de una etiqueta o de varias dentro del índice (-1 si no está).
`idx.slice_indexer(start, end) · idx.slice_locs(start, end) · idx.get_slice_bound(label, side)`	Posiciones de corte para rebanadas por etiqueta.
`idx.isin(values) · idx.where(cond, other) · idx.putmask(mask, value)`	Pertenencia y reemplazo condicional.
`idx.union(other) · idx.intersection(other) · idx.difference(other) · idx.symmetric_difference(other)`	Operaciones de conjuntos entre índices.
`idx.append(other) · idx.insert(loc, item) · idx.delete(loc) · idx.drop(labels)`	Agregar, insertar y quitar etiquetas.
`idx.unique() · idx.drop_duplicates() · idx.duplicated() · idx.nunique() · idx.value_counts()`	Únicos, repetidos y frecuencias.
`idx.sort_values() · idx.argsort() · idx.sortlevel() · idx.searchsorted(value)`	Ordenamiento y búsqueda ordenada.
`idx.min() · idx.max() · idx.argmin() · idx.argmax()`	Extremos y sus posiciones.
`idx.map(mapper) · idx.astype(dtype) · idx.round(decimals) · idx.shift(periods) · idx.diff()`	Transformaciones.
`idx.fillna(value) · idx.dropna() · idx.isna() · idx.notna() · idx.hasnans`	Manejo de faltantes.
`idx.equals(other) · idx.identical(other) · idx.is_(other)`	Igualdad de valores, igualdad con mismos atributos e identidad del objeto.
`idx.join(other, how, level) · idx.reindex(target) · idx.groupby(values)`	Unir, reindexar y agrupar.
`idx.take(indices) · idx.repeat(repeats) · idx.copy() · idx.ravel() · idx.transpose()`	Selección por posición y copias.

Elemento	Qué hace
`idx.droplevel(level)` · `idx.get_level_values(level)` · `idx.nlevels` · `idx.to_flat_index()`	Niveles y aplanado de un índice jerárquico.
`idx.factorize()` · `idx.memory_usage(deep)` · `idx.infer_objects()` · `idx.item()` · `idx.view()`	Codificación, memoria y utilidades.
`idx.str`	Accesor de texto sobre un índice de texto (como Series.str).

MultiIndex (índice jerárquico)

Elemento	Qué hace
`pd.MultiIndex.from_arrays(arrays, names)`	Crea el índice desde una lista de arreglos (uno por nivel).
`pd.MultiIndex.from_tuples(tuples, names)`	Crea el índice desde una lista de tuplas.
`pd.MultiIndex.from_product(iterables, names)`	Crea todas las combinaciones de varias listas.
`pd.MultiIndex.from_frame(df, names)`	Crea el índice desde las columnas de un DataFrame.
`mi.levels` · `mi.codes` · `mi.levshape` · `mi.dtypes` · `mi.nlevels`	Valores de cada nivel, códigos, cantidad de valores por nivel y tipos.
`mi.get_level_values(level)` · `mi.droplevel(level)` · `mi.swaplevel(i, j)` · `mi.reorder_levels(order)`	Obtener, quitar, intercambiar y reordenar niveles.
`mi.set_levels(levels)` · `mi.set_codes(codes)` · `mi.remove_unused_levels()`	Redefinir los niveles o códigos, y descartar niveles no usados.
`mi.get_loc_level(key, level)` · `mi.get_locs(seq)`	Posiciones de las filas que corresponden a una clave parcial o a una secuencia de claves.
`mi.truncate(before, after)` · `mi.equal_levels(other)`	Recorte por rango y comparación de niveles.
`df.loc[(a, b), col]` · `df.loc[pd.IndexSlice[:, b], :]` · `df.xs(key, level)`	Formas de seleccionar en un índice jerárquico.

DatetimeIndex (índice de fechas)

Elemento	Qué hace
`dti.year · month · day · hour · dayofweek · quarter · dayofyear · days_in_month`	Componentes de cada fecha.
`dti.is_month_start / is_month_end / is_quarter_start / is_quarter_end / is_year_start / is_year_end / is_leap_year`	Indicadores de calendario.
`dti.day_name() · dti.month_name() · dti.strftime(format)`	Nombres y formato de texto.
`dti.normalize() · dti.floor(freq) · dti.ceil(freq) · dti.round(freq) · dti.snap(freq)`	Redondeos.
`dti.tz · dti.tz_localize(tz) · dti.tz_convert(tz)`	Zonas horarias.
`dti.freq · dti.freqstr · dti.inferred_freq · dti.is_normalized · dti.resolution · dti.unit · dti.as_unit(unit)`	Frecuencia, resolución y unidad interna.
`dti.to_period(freq) · dti.to_pydatetime() · dti.to_julian_date() · dti.isocalendar()`	Conversiones.
`dti.indexer_at_time(time) · dti.indexer_between_time(start, end)`	Posiciones de las filas a una hora o entre dos horas.
`dti.mean() · dti.std()`	Fecha promedio y desvío en duración.

25. Visualización (plot)

pandas dibuja gráficos usando matplotlib, que debe estar instalado. df.plot(...) devuelve un objeto de ejes de matplotlib; para mostrarlo o guardarlo se usa plt.show() o plt.savefig(...).

DataFrame.plot / Series.plot

Elemento	Qué hace
`df.plot(kind=«line», x=None, y=None, ax=None, ...)`	Método general. kind: «line», «bar», «barh», «hist», «box», «kde» (o «density»), «area», «pie», «scatter» y «hexbin». x e y eligen columnas (en scatter son obligatorios).
`df.plot.line() · .bar() · .barh() · .hist(bins) · .box() · .kde() · .area() · .pie() · .scatter(x, y, s, c) · .hexbin(x, y, gridsize)`	Atajos con un método por tipo de gráfico.
`subplots · layout · sharex · sharey`	Dibujar cada columna en su propio gráfico, disposición en cuadrícula y ejes compartidos.
`figsize · title · grid · legend · style · colormap · colorbar · fontsize · rot`	Tamaño de la figura, título, cuadrícula, leyenda, estilo de línea, mapa de colores, barra de color, tamaño de letra y rotación de etiquetas.
`xlim · ylim · xticks · yticks · logx · logy · loglog`	Límites de ejes, marcas y escalas logarítmicas.
`stacked · secondary_y · yerr · xerr · table · use_index · position · mark_right · include_bool`	Barras o áreas apiladas, eje secundario, barras de error, tabla bajo el gráfico, uso del índice como eje, posición de las barras y otras opciones.
`backend`	Biblioteca de dibujo a usar (por defecto matplotlib; existen backends como plotly).
`df.hist(column, by, bins, ...)`	Histogramas de todas las columnas numéricas (o agrupados con by).
`df.boxplot(column, by, ...)`	Diagramas de caja.
`gb.plot · gb.hist() · gb.boxplot()`	Gráficos por grupo.

pd.plotting (gráficos especiales)

Elemento	Qué hace
`scatter_matrix(frame, alpha, figsize, diagonal, ...)`	Matriz de gráficos de dispersión entre todas las columnas.
`parallel_coordinates(frame, class_column, ...) · andrews_curves(frame, class_column) · radviz(frame, class_column)`	Visualizaciones de datos multivariados por clase.
`autocorrelation_plot(series) · lag_plot(series, lag) · bootstrap_plot(series, ...)`	Gráficos para series temporales y de remuestreo.
`table(ax, data)`	Dibuja una tabla dentro de un gráfico.
`plot_params · register_matplotlib_converters() · deregister_matplotlib_converters()`	Parámetros de dibujo y conversores de fechas de pandas para matplotlib.

26. Estilos visuales (df.style)

df.style devuelve un objeto Styler: una vista con formato condicional que se puede mostrar en un notebook, exportar a HTML o guardar a Excel. No modifica los datos.

Elemento	Qué hace
`styler.format(formatter=None, subset=None, na_rep=None, precision=None, decimal=«.», thousands=None, escape=None, hyperlinks=None)`	Formato de visualización de los valores: decimales, separador de miles, símbolo, texto para faltantes.
`styler.format_index(...) · styler.format_index_names(...) · styler.relabel_index(labels)`	Formato y nombres visibles de las etiquetas de filas y columnas.
`styler.apply(func, axis=0, subset=None, **kwargs)`	Aplica una función que devuelve estilos CSS por columna, fila o tabla completa.
`styler.map(func, subset=None)`	Aplica una función que devuelve un estilo CSS para cada celda (reemplaza a applymap).
`styler.apply_index(func) · styler.map_index(func)`	Igual, pero sobre las etiquetas de filas o columnas.
`styler.highlight_max(subset, color, axis) · highlight_min(...) · highlight_null(...)`	Resalta el máximo, el mínimo o los faltantes.
`styler.highlight_between(left, right, ...) · highlight_quantile(q_left, q_right, ...)`	Resalta valores dentro de un rango o de un rango de cuantiles.
`styler.background_gradient(cmap, low, high, axis, subset, text_color, vmin, vmax, gmap)`	Colorea el fondo según el valor (mapa de calor).
`styler.text_gradient(...)`	Igual, pero colorea el texto.
`styler.bar(subset, axis, color, width, align, vmin, vmax)`	Barras de datos dentro de las celdas.
`styler.set_properties(subset, **kwargs)`	Aplica propiedades CSS fijas a celdas (alineación, color, borde).
`styler.set_caption(caption)`	Título de la tabla.
`styler.set_table_styles(styles) · set_table_attributes(attributes) · set_td_classes(classes) · set_uuid(uuid)`	Estilos CSS generales de la tabla, atributos HTML y clases.
`styler.set_tooltips(ttips) · styler.set_sticky(axis)`	Globos de ayuda al pasar el mouse y encabezados fijos al desplazar.
`styler.hide(subset, axis, level, names)`	Oculta filas, columnas o el índice.
`styler.concat(other)`	Agrega una fila de totales u otra tabla debajo.
`styler.pipe(func) · styler.clear() · styler.export() · styler.use(styles)`	Encadenar funciones, quitar estilos, y exportar o reutilizar un conjunto de estilos.
`styler.to_html(...) · styler.to_latex(...) · styler.to_string(...) · styler.to_typst(...)`	Exporta la tabla con estilos a HTML, LaTeX, texto o Typst.
`styler.to_excel(excel_writer, sheet_name, ...)`	Guarda en Excel conservando colores y formatos.

27. Opciones de configuración

Controlan cómo se muestran los datos y algunos comportamientos. Se consultan con nombres de la forma «display.max_rows».

Funciones

Elemento	Qué hace
`pd.set_option(pat, value)`	Cambia una opción, p. ej. pd.set_option(«display.max_columns», None) para mostrar todas las columnas.
`pd.get_option(pat)`	Consulta el valor actual.
`pd.reset_option(pat)`	Vuelve al valor por defecto (acepta un patrón: «display» las reinicia todas).
`pd.describe_option(pat)`	Muestra la descripción de una opción.
`pd.option_context(pat, value, ...)`	Administrador de contexto: cambia opciones solo dentro de un bloque with.
`pd.options.display.max_rows = n`	Forma alternativa de acceso, como atributos.
`pd.set_eng_float_format(accuracy, use_eng_prefix)`	Muestra los números en notación de ingeniería.

Opciones de visualización (display.*)

Elemento	Qué hace
`display.max_rows` · `display.min_rows`	Máximo de filas a mostrar y cuántas se muestran cuando se trunca.
`display.max_columns` · `display.width` · `display.expand_frame_repr`	Máximo de columnas, ancho de la salida y si se permite que una tabla ancha se muestre en varios bloques.
`display.max_colwidth`	Ancho máximo del texto de cada celda.
`display.precision` · `display.float_format`	Cantidad de decimales y función de formato de los números, p. ej. «{:,.2f}».format.
`display.show_dimensions` · `display.colheader_justify` · `display.chop_threshold`	Mostrar la forma al pie, alineación de encabezados y umbral bajo el cual un número se muestra como 0.
`display.large_repr` · `display.max_info_columns` · `display.max_info_rows` · `display.memory_usage` · `display.max_seq_items`	Comportamiento de la vista de tablas grandes, de info() y de listas largas.
`display.date_dayfirst` · `display.date_yearfirst` · `display.unicode` · `display.multi_sparse`	Interpretación de fechas ambiguas, ancho de caracteres Unicode y cómo se muestran los niveles repetidos de un MultiIndex.

Otras (mode.*)

Elemento	Qué hace
`mode.string_storage`	Almacenamiento del tipo string: «python» o «pyarrow».
`mode.performance_warnings`	Activa o desactiva las advertencias de rendimiento.
`mode.chained_assignment`	Cómo avisar ante asignaciones encadenadas.
`mode.copy_on_write`	Obsoleta en pandas 3: el copy-on-write está siempre activo y esta opción ya no tiene efecto.

28. Errores y advertencias (pd.errors)

Excepciones y avisos propios de pandas. Los más comunes:

Elemento	Qué hace
`EmptyDataError`	Se intentó leer un archivo vacío o sin columnas.
`ParserError · ParserWarning`	Falló la lectura de un archivo por formato (p. ej. una línea con más campos de lo esperado).
`DtypeWarning`	Una columna tiene tipos mezclados al leer por bloques; se arregla con dtype explícito o low_memory=False.
`MergeError`	Error en merge, p. ej. al incumplir la relación indicada en validate.
`ChainedAssignmentError`	Se intentó asignar mediante una selección encadenada, que ya no modifica el original en pandas 3.
`IndexingError`	Indexación inválida (p. ej. demasiados indexadores).
`InvalidIndexError`	Se usó una etiqueta no válida como clave.
`UnsortedIndexError`	Se usó una rebanada sobre un MultiIndex sin ordenar; se arregla con sort_index().
`OutOfBoundsDatetime · OutOfBoundsTimedelta`	La fecha o duración está fuera del rango representable.
`IntCastingNaNError`	Se intentó convertir a entero una columna que contiene NaN.
`DuplicateLabelError`	Operación que produciría etiquetas repetidas con allows_duplicate_labels=False.
`SpecificationError`	Especificación inválida en agg (p. ej. una agregación con nombre mal formada).
`DataError`	No hay datos numéricos sobre los que operar.
`NullFrequencyError`	Se pidió una operación que necesita frecuencia en un índice de fechas sin ella.
`UndefinedVariableError · NumExprClobberingError`	Variable inexistente o con nombre conflictivo en query/eval.
`PerformanceWarning`	Aviso de operación que puede ser muy lenta.
`OptionError · AbstractMethodError · InvalidComparison · LossySetitemError · IncompatibleFrequency`	Errores de opciones, de métodos sin implementar, de comparaciones inválidas, de asignaciones con pérdida y de frecuencias incompatibles.
`PandasChangeWarning · PandasDeprecationWarning · PandasFutureWarning · PandasPendingDeprecationWarning · Pandas4Warning · Pandas5Warning`	Avisos sobre funciones que cambiarán o desaparecerán en próximas versiones (4 y 5).
`DatabaseError · ClosedFileError · PyperclipException · ValueLabelTypeMismatch · PossibleDataLossError · InvalidColumnName · UnsupportedFunctionCall`	Errores específicos de SQL, HDF5, portapapeles, Stata y otras funciones de entrada y salida.

29. Cambios importantes de pandas 3.0

Muchos tutoriales y respuestas viejas usan código de pandas 1.x o 2.x que ya no funciona. Esta guía está basada en pandas 3.0.

Elemento	Qué hace
`Copy-on-Write siempre activo`	Cada objeto derivado se comporta como una copia independiente. Las asignaciones encadenadas (df[df.a > 0][«b»] = 1 o df[«a»][0] = 1) ya no modifican el original; hay que usar df.loc[fila, columna] = valor. Desapareció SettingWithCopyWarning.
`Texto como tipo str`	Las columnas de texto se infieren como tipo str (no object). El faltante sigue siendo NaN.
`Resolución de fechas`	Las columnas de fecha ya no son siempre datetime64[ns]: la unidad (s, ms, us, ns) se infiere de los datos; con texto suele resultar microsegundos.
`Alias de frecuencia`	Se eliminaron «M», «Q», «Y», «A», «H», «T» y «S». Usar «ME», «QE», «YE», «h», «min» y «s».
`Eliminados`	DataFrame.append (usar pd.concat), DataFrame.applymap (usar DataFrame.map), fillna(method=...) (usar ffill y bfill), el parámetro axis de groupby, date_parser e infer_datetime_format de las lecturas, e iteritems (usar items).
`groupby y pivot_table`	El parámetro observed pasó a True por defecto: las categorías sin datos ya no aparecen en el resultado.
`stack`	Usa la nueva implementación por defecto (future_stack=True).
`Parámetro copy`	Quedó obsoleto en la mayoría de los métodos: ya no hace falta, porque Copy-on-Write evita copias innecesarias.
`pd.col`	Novedad: expresiones sobre columnas para usar en assign y selecciones.

30. Trampas frecuentes

Comportamientos de pandas que suelen sorprender al empezar.

- Los operadores lógicos son & | ~ y cada condición va entre paréntesis: df[(df.a > 1) & (df.b < 5)]. Usar and / or da error.
- loc incluye el extremo final de una rebanada y iloc no lo incluye.
- Casi todos los métodos devuelven un objeto nuevo. Si no reasignás el resultado (df = df.dropna()), el original no cambia. Con inplace=True devuelven None, así que df = df.dropna(inplace=True) deja df en None.
- Una columna de enteros con valores faltantes pasa a float64. Para mantener enteros, usar el tipo Int64 (convert_dtypes o astype).
- df.to_excel y df.to_csv escriben el índice como primera columna; si no lo querés, pasá index=False.
- value_counts, groupby y nunique ignoran los faltantes por defecto (dropna=True).
- Las operaciones entre objetos se alinean por etiquetas, no por posición; si los índices no coinciden aparecen NaN.
- apply(axis=1), iterrows y bucles for sobre filas son lentos; casi siempre existe una operación vectorizada equivalente.
- Al leer CSV de origen latino, revisar sep, decimal, thousands y encoding (p. ej. «;», «,», «.» y «latin-1»); y parse_dates con dayfirst=True para fechas día/mes/año.
- Un merge con claves repetidas multiplica las filas. Usá validate y revisá df.shape antes y después.
- Los nombres de columna con espacios o símbolos no se pueden usar como df.nombre; hay que escribir df[«nombre con espacios»].
- axis=0 opera por columnas (de arriba hacia abajo) y axis=1 por filas (de izquierda a derecha); confunde a casi todos al principio.

- Para leer Excel hace falta openpyxl (o el motor adecuado) instalado, y read_excel no lee fórmulas sino el último valor guardado.